

Ejercicio 1: Sistema de Gestión de Pedidos

Desarrolla un pequeño sistema orientado a objetos para gestionar el estado de los pedidos de una tienda online. Utilizarás un **Enum Puro** para los estados, organizarás el código en `namespaces` y usarás un **alias** para importar una clase de utilidad.

1. **Estructura del Proyecto:**
   Crea la siguiente estructura de directorios y archivos:

```
ProyectoUT03_71/
├── App/
│   ├── Enums/
│   │   └── EstadoPedido.php
│   └── Models/
│       └── Pedido.php
├── Utils/
│   └── Formatter.php
└── index.php
```

2. **El Enum `EstadoPedido` (Enum Puro):**
   - Debe estar en el namespace `App\Enums`.
   - Define un `enum` puro llamado `EstadoPedido` con los siguientes casos: `Pendiente`, `Enviado`, `Entregado`, `Cancelado`.
   - Añade un método público `descripcion(): string` que devuelva un texto explicativo para cada estado (ej: "El pedido ha sido enviado.")

3. **La Clase de Utilidad `Formatter`:**
   - Debe estar en el namespace `Utils`.
   - Crea una clase llamada `Formatter` con un único método estático público: `aMayusculas(string $texto): string`, que devuelva el texto recibido en mayúsculas.

4. **La Clase Principal `Pedido`:**
   - Debe estar en el namespace `App\Models`.
   - La clase debe importar `App\Enums\EstadoPedido`.
   - Debe importar `Utils\Formatter` **usando el alias `TextFormatter`**.
   - Tendrá dos propiedades públicas: `id` (entero) y `estado` (del tipo `EstadoPedido`).

- Su constructor `__construct(int $id)` recibirá un ID y deberá inicializar el estado del pedido a `Pendiente` por defecto.
- Tendrá un método público `actualizarEstado(EstadoPedido $nuevoEstado)` para cambiar el estado actual.
- Tendrá un método público `obtenerInfo(): string` que devuelva una cadena formateada con el ID y el nombre del estado (ej: "PEDIDO ID: 101 | ESTADO: PENDIENTE"). **Este método debe usar la clase `TextFormatter`** para convertir el resultado a mayúsculas.

5. **Punto de Entrada `index.php`:**
   - Importa las clases que necesites.
   - En este archivo, escribe el código para:
     1. Crear un nuevo objeto `Pedido` con un ID de tu elección.
     2. Mostrar por pantalla su información inicial y la descripción de su estado.
     3. Actualizar el estado del pedido a `Enviado` y volver a mostrar su información y descripción.
     4. Finalmente, actualizar el estado a `Entregado` y mostrar la información por última vez.

Ejercicio 2: Clasificación de Documentos de una Biblioteca

Crea un sistema para clasificar y gestionar documentos en una biblioteca digital. El tipo de documento debe poder ser almacenado como un texto simple (ej. en una base de datos), por lo que utilizarás un **Backed Enum**. El sistema también usará `namespaces` y un **alias**.

Requisitos:

1. **Estructura del Proyecto:**
   Crea la siguiente estructura de directorios y archivos:

```
ProyectoUT03_72/
├── Biblioteca/
│   ├── Enums/
│   │   └── TipoDocumento.php
│   └── Logica/
│       └── Documento.php
├── Almacenamiento/
│   └── BaseDeDatos.php
└── index.php
```

2. **El Enum `TipoDocumento` (Backed Enum):**
   - Debe estar en el namespace `Biblioteca\Enums`.
   - Define un `enum` llamado `TipoDocumento` respaldado por el tipo `string` (`Backed Enum`).
   - Tendrá los siguientes casos con sus valores correspondientes:
     - `Libro` con el valor `'book'`.
     - `Revista` con el valor `'magazine'`.
     - `Tesis` con el valor `'thesis'`.
   - Añade un método público `nivelAcceso(): string` que devuelva "Acceso General" para `Libro` y `Revista`, y "Acceso Restringido" para `Tesis`.
3. **La Clase Simuladora `BaseDeDatos`:**
   - Debe estar en el namespace `Almacenamiento`.
   - Crea una clase `BaseDeDatos` para simular la interacción con una BBDD.
   - Debe tener un único método estático público `guardar(string $tabla, array $datos)` que simplemente imprima en pantalla un mensaje indicando qué datos se están "guardando" (ej: "Guardando en tabla 'documentos': titulo=..., tipo_documento=...").
4. **La Clase Principal `Documento`:**
   - Debe estar en el namespace `Biblioteca\Logica`.
   - La clase debe importar `Biblioteca\Enums\TipoDocumento`.

- Debe importar `Almacenamiento\BaseDeDatos` **usando el alias `GestorDB`**.
- Tendrá dos propiedades públicas: `titulo` (string) y `tipo` (del tipo `TipoDocumento`).
- Su constructor `__construct(string $titulo, TipoDocumento $tipo)` recibirá ambos valores.
- Tendrá un método público `guardar(): void`. Este método debe llamar a `GestorDB::guardar()`, pasándole como datos el título del documento y el **valor (`->value`)** del `enum tipo`.
- Tendrá un método público `obtenerInfoDetallada(): string` que devuelva una cadena con el título, el nombre del tipo de documento y el nivel de acceso (ej: "**PHP para Principiantes** (Libro) - Acceso General").

5. **Punto de Entrada `index.php`:**
   - Importa las clases que necesites.
   - Escribe el código para:
     1. Crear un `Documento` de tipo `Libro`, mostrar su información detallada y llamar a su método `guardar()`.
     2. Crear un `Documento` de tipo `Tesis`, mostrar su información y guardarlo.
     3. Demostrar la funcionalidad `from()` de los Backed Enums:
        - Crea una variable con el string `'magazine'`.
        - Usa `TipoDocumento::from()` para convertir ese string en un caso del enum.
        - Crea un nuevo `Documento` con el resultado, muestra su información y guárdalo.

Ejercicio 3: Saludo Personalizado con `$_GET`

Concepto Clave: `$_GET` se utiliza para recoger datos que se envían a través de la URL (en la "query string"). Es visible para el usuario e ideal para datos no sensibles como términos de búsqueda, IDs de productos o filtros.

Enunciado

Crea una página web que muestre un saludo personalizado. La página debe tener un formulario con un solo campo de texto para que el usuario introduzca su nombre y un botón de envío.

1. El formulario debe usar el método **`GET`**.
2. Cuando el usuario envíe su nombre, la misma página debe recargarse y mostrar un mensaje de bienvenida, por ejemplo: "¡Hola, Carlos!".
3. Observa cómo, después de enviar, el nombre aparece en la URL del navegador (ej: `saludo_get.php?nombre=Carlos`).

Solución

Crea un archivo llamado formulario.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Ejercicio con GET</title>
    <style>
        body { font-family: sans-serif; text-align: center; margin-top:
50px; }
        form { margin-top: 20px; }
        h1 { color: #333; }
    </style>
</head>
<body>

    <h1>Formulario de saludo</h1>
            <!-- Este es el formulario. Su método es "get" -->
    <form action="saludo_get.php" method="get">
        <label for="nombre">Introduce tu nombre:</label>
        <input type="text" id="nombre" name="nombre">
        <input type="submit" value="Saludar">
    </form>

</body>
</html>
```

Crea un archivo llamado `saludo_get.php`

```php
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Saludo</title>
</head>
<body>
    <?php
// --- LÓGICA DE PHP ---

// 1. Verificamos si el parámetro 'nombre' existe en la URL y no está
vacío.
//    Usamos isset() para evitar errores si la variable no ha sido
enviada.
if (isset($_GET['nombre']) && !empty($_GET['nombre'])) {

    // 2. Si existe, guardamos el nombre en una variable.
    //    Usamos htmlspecialchars() como medida de seguridad básica para
evitar
    //    que se pueda inyectar HTML o JavaScript malicioso.
    $nombre_usuario = htmlspecialchars($_GET['nombre']);

} else {
    // 3. Si no se envió ningún nombre, usamos un valor por defecto.
    $nombre_usuario = "invitado";
}


// 4. Creamos el mensaje de saludo completo.
$mensaje_saludo = "¡Hola, " . $nombre_usuario . "!";
echo "<h1> $mensaje_saludo</h1>";

?>
</body>
</html>
```

Ejercicio 2: Mini Calculadora con `$_POST`

Concepto Clave: `$_POST` recoge datos enviados en el cuerpo de la petición HTTP. No son visibles en la URL, lo que lo hace ideal para formularios con información sensible (contraseñas) o con muchos datos.

Enunciado

Desarrolla una calculadora simple que sume dos números. La página mostrará un formulario con dos campos numéricos y un botón para sumar.

1. El formulario debe usar el método **`POST`**.
2. Cuando el usuario envíe los dos números, la misma página debe procesarlos, calcular la suma y mostrar el resultado claramente, por ejemplo: "El resultado de sumar 5 y 10 es 15".
3. Fíjate que, al contrario que con GET, los números que envías no aparecen en la URL.

Solución del Ejercicio 2

Crea un archivo llamado formulario_calculadora.html

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Ejercicio con POST</title>
    <style>
        body { font-family: sans-serif; text-align: center; margin-top:
50px; }
        form { margin-top: 20px; }
    </style>
</head>
<body>

    <h1>Mini Calculadora</h1>

    <!-- Este formulario usa el método "post" -->
    <form action="calculadora_post.php" method="post">
        <input type="number" name="num1" placeholder="Primer número"
required>
        <span>+</span>
        <input type="number" name="num2" placeholder="Segundo número"
required>
```

```
        <br><br>
        <input type="submit" value="Sumar">
    </form>


</body>
</html>
```

Crea un archivo llamado `calculadora_post.php`.

```
<?php
// procesa post.php - ejemplo sencillo para mostrar $_POST
header('Content-Type: text/html; charset=utf-8');
?>
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Calculadora</title>
</head>
<body>
    <?php
// --- LÓGICA DE PHP ---

// 1. Inicializamos la variable que contendrá el resultado para que no dé
error al cargar la página por primera vez.
$resultado = "";

// 2. La mejor forma de saber si un formulario POST ha sido enviado es
comprobar el método de la petición.
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // 3. Verificamos que ambos números han sido enviados y no están
vacíos.
    if (isset($_POST['num1']) && isset($_POST['num2']) && $_POST['num1']
!== '' && $_POST['num2'] !== '') {

        // 4. Obtenemos los números y los convertimos a un tipo numérico
(float para admitir decimales).
        $num1 = (float)$_POST['num1'];
        $num2 = (float)$_POST['num2'];
```

```
        // 5. Realizamos la suma.
        $suma = $num1 + $num2;

        // 6. Preparamos el mensaje de resultado para mostrarlo después
en el HTML.
       echo $resultado = "<h2>El resultado de sumar $num1 y $num2 es:
$suma</h2>";
    } else {
       echo $resultado = "<p style='color: red;'>Por favor, introduce
ambos números.</p>";
    }
}
?>

</body>
</html>
```